

OBJETIVOS

- Familiarizar o aluno com o SO MINIX e testes em ambiente de máquina virtual.
 - Compilar e modificar o kernel
- Criação de processos, IPC, gerenciamento de sinais
 - Processos órfãos e zumbis
- Gerenciamento de I/O no MINIX
 - Criar uma chamada de sistema;
 - Criar uma chamada de *kernel* e usá-la via uma chamada de sistemas (item anterior);
 - Criar um *character device*.

TAREFA 7: criando uma chamada de sistema (de 10/03/2025 a 13/03/2025)

TEÓRICO: No MINIX 3 servidores tratam as chamadas de sistema. Conforme visto na unidade 1.1 (Introdução ao MINIX), existem servidores para lidar com diversos aspectos do sistema operacional (gerenciamento de processos, gerenciamento de sistema de arquivos, gerenciamento conexões de rede etc). Para adicionar uma nova chamada de sistema são necessárias duas etapas: escrever um manipulador (*handler*) de chamada de sistema e escrever uma biblioteca de usuário. O manipulador de chamada de sistema é uma função que é chamada em resposta a um usuário solicitando uma chamada de sistema. Cada chamada de sistema tem um manipulador. Uma biblioteca do usuário empacota os parâmetros para a chamada de sistema e chama o manipulador no servidor apropriado. Um usuário sempre invoca uma chamada de sistema usando a biblioteca de usuário.

O manipulador de chamadas do sistema deve ser colocado em um servidor apropriado, que por sua vez irá processar uma solicitação do usuário, invocando o manipulador correspondente. É importante escolher o servidor correto para a inclusão da chamada de sistema. Por exemplo, se a chamada de sistema deve atualizar o sistema de arquivo ou estruturas de dados `fproc`, então o manipulador de chamada de sistema deve ser colocado no servidor FS (no MFS, para sistema de arquivos).

Para esta prática, o aluno irá adicionar uma chamada de sistema simples no MINIX 3, no

servidor PM (gerenciador de processo), a qual irá imprimir a seguinte mensagem: “Eu sou uma chamada de sistema!”.

Para auxiliar o aluno, esta prática será dividida em algumas etapas – estas etapas devem constar no relatório:

- (a) O código fonte para todos os servidores está localizado em `/usr/src/servers`. Cada servidor possui um diretório separado. O gerenciador de processos (PM) está localizado em `/usr/src/servers/pm`. Cada diretório de servidor possui dois arquivos chave: `table.c` e `proto.h`. Estes arquivos contêm a definição para a tabela `call_vec`. A tabela `call_vec` é um vetor de ponteiros de função que é indexado pelo número da chamada de sistema. Em cada linha, o endereço de um manipulador de chamada de sistema é atribuído para uma entrada na tabela e o índice da entrada é o número da chamada de sistema. **Modifique os arquivos `table.c` e `proto.h` de modo a incluir o manipulador `do_mycall`. Deduza como as entradas para a criação deste manipulador devem ser inseridas nos arquivos mencionados. Use a entrada de número 70 para identificar esta chamada de sistema. Observe que `do_mycall` não recebe parâmetros de entrada. Esta chamada irá apenas imprimir a mensagem: “Eu sou uma chamada de sistema!”. Em `proto.h`, insira o protótipo de sua chamada de sistema na seção destinada ao arquivo `misc.c`.**
- (b) A implementação da chamada de sistema deve aparecer em `/usr/src/servers/pm`. Esta pode ser implementada em um arquivo “.c” a parte, porém, para simplificar o processo, iremos inserir sua implementação em `/usr/src/servers/pm/misc.c`. O corpo da chamada deverá conter apenas:

```
printf("Eu sou uma chamada de sistema!");  
return(OK);
```

Para definir o cabeçalho da chamada de sistema, observe o cabeçalho da função `do_procstat`. Ambos são similares.
- (c) Vá ao diretório `/usr/src/servers` e faça um `make`. Isso deverá mostrar erros de compilação caso você tenha realizado alguma alteração incorretamente. Corrija seus erros caso seja necessário.
- (d) É hora de mapear a chamada de sistema e seu número (70). Para tanto, você deve ir para o diretório `/usr/src/include/minix` e editar o arquivo `callnr.h` de modo a inserir

o mapeamento para a sua chamada de sistema. Chame-a de **MYCALL** no contexto desse arquivo.

- (e) **Para simplificar o procedimento de criação da biblioteca de usuário**, crie em **/usr/src/include** o arquivo **mylib.h**. Este arquivo irá executar efetivamente a chamada de sistema criada. O código a ser inserido nessa biblioteca é:

```
#include<lib.h>
#define mycall      _mycall
#include<unistd.h>
<<similar ao cabeçalho disponível em (b)>>
{
    message m;
    return (_syscall(PM_PROC_NR, MYCALL, &m));
}
```

Não se esqueça de incluir uma entrada para **mylib.h** em **/usr/src/include/Makefile**.

- (f) Você observou no código da questão anterior a seguinte declaração: “message m”. Explique em que consiste esse tipo de dado. Explique também o papel da chamada de sistema **_syscall()** (obs. Os slides da unidade 2 possuem uma referência a esta questão. Para maiores detalhes consulte o livro Projeto de Sistemas Operacionais que consta na referência do curso).
- (g) Para finalizar o processo de criação de sua chamada de sistema, vá ao diretório **/usr/src/tools** compile o kernel, bibliotecas e includes e instale no HD.
- (h) Crie um programa de teste para sua nova chamada de sistema. **Dica:** você teria dificuldade em demonstrar a chamada de sistema **printf()**? O raciocínio é análogo.